

# Course Outline: Caching

---

**Title:** Caching: Faster Responses Without Extra Work **Skill:** Skill 37 — Optimizar la arquitectura para alto tráfico empleando Caching **Learnpack:** + Caching **File:** s37-w13d37-caching **Prerequisite:** Serializers (s36-w12d36-serializers) **Target Audience:** Beginner developers building FastAPI backends who understand serializers and know how to shape API responses **Total Lessons:** 11 **Format:** Mixed (9% hands-on = 1 lesson)

---

## Course Description

Students already know how to return the right data in the right shape. But their API still recalculates the same result on every request — even when nothing has changed. This course introduces caching: a technique for storing the result of expensive operations so they can be reused without recomputing. Students will learn when caching helps, where to put the cache, how to keep it fresh, and which of the four established strategies to apply in a given scenario.

---

## Course Philosophy

This course emphasizes:

- Understanding the problem before reaching for the solution — caching is a tradeoff, not a default
  - Building intuition for the cost/freshness spectrum rather than memorizing rules
  - Starting with the simplest implementation (in-memory dict) before introducing infrastructure (Redis)
- 

## Course Statistics Summary

| Section                   | Lessons   |
|---------------------------|-----------|
| 00 - Welcome              | 1         |
| 01 - Caching Fundamentals | 2         |
| 02 - Cache Architecture   | 3         |
| 03 - Caching Strategies   | 3         |
| 04 - Assessment           | 1         |
| 05 - Outro                | 1         |
| <b>Total</b>              | <b>11</b> |

---

## Learning Objectives:

- Understand what problem caching solves and why it matters at scale
- Get a preview of the four sections that build from concept to implementation

## Content Outline:

- The performance problem: your API is correct but slow — same query, 50ms, 200 times a second
- Caching as a solution: store the result once, reuse it many times
- What this course covers: fundamentals → architecture → invalidation → strategies → practice

**Transitions:** Section 01 starts by defining what caching actually is and how it works mechanically.

## Key Principles:

- Caching trades memory for speed
- The hardest part of caching is not storing — it's knowing when to stop storing

## Examples:

- A `/products` endpoint that queries the database on every request vs. one that caches the result for 60 seconds
  - A user profile endpoint that hits the DB 500 times a minute for data that changes once a day
- 

## 01.0 Caching Fundamentals

### Learning Objectives:

- Define what a cache is and how it differs from a database
- Explain the cache hit / miss cycle
- Understand what "memory cache" means and where it sits in the request flow

### Content Outline:

- What caching is: storing the result of an expensive operation so it can be reused
- The memory cache: a fast-access store (dict, Redis) between the requester and the expensive source
- The hit/miss cycle: check cache → hit: return immediately → miss: compute, store, return
- What counts as "expensive": DB queries, external API calls, heavy computations

**Transitions:** Now that we know what caching is and how the cycle works, the next lesson asks the harder question: when is it actually worth caching?

### Key Principles:

- A cache is a copy — the original source of truth is always elsewhere
- Every cache hit saves a round trip to something slower
- Caching only helps when the same result is requested more than once

### Examples:

- Requesting `/users/42` 100 times in one minute: without cache → 100 DB queries; with cache → 1 DB query + 99 instant returns
  - The hit/miss cycle visualized as: request → check dict → found? return it : query DB, save to dict, return it
- 

## 01.1 When to Cache

### Learning Objectives:

- Apply the cost-of-computation vs. cost-of-storage tradeoff to decide what to cache
- Identify when freshness concerns outweigh the speed benefit
- Recognize data that must never be placed in a shared cache

### Content Outline:

- Cost of computation vs. cost of storage: is the operation expensive enough to justify the memory?
- High read frequency: if the same result is requested many times, caching pays off fast
- Freshness vs. speed: data that changes constantly loses its caching benefit
- What NOT to cache: per-user private data in a shared cache, highly volatile data, one-time queries

**Transitions:** Once we know what's worth caching, the next question is: where does the cache actually live? Section 02 maps out the options.

### Key Principles:

- Cache what is expensive to compute and cheap to store
- Cache what is read many times and written rarely
- Never cache private or authenticated data in a shared store

### Examples:

- Worth caching: a product catalog that changes once a day but is queried thousands of times
  - Not worth caching: a real-time stock price that changes every second
  - **Anti-pattern — Shared Cache for Private Data:** Storing a user's shopping cart in a shared CDN cache. Another user's request matches the cache key and receives someone else's cart. The fix: private data belongs in per-user stores (session, Redis with user-scoped keys) — never in a shared public cache.
- 

## 02.0 Cache Architecture

### Learning Objectives:

- Understand that "cache" is not one thing — it exists at multiple layers with different tradeoffs
- Distinguish between in-process, distributed, and HTTP-level caching

### Content Outline:

- The spectrum: in-process memory (fastest, local) → distributed cache (shared, networked) → CDN (global, HTTP-level)
- Each layer trades speed for reach: in-process is fastest but not shared; CDN is global but coarse
- The architecture question: how many instances of your app are running, and do they need a shared cache?

**Transitions:** 02.1 covers each cache type in detail — what it is, when to use it, and what breaks if you choose wrong. 02.2 then covers the hardest architectural problem: keeping the cache up to date.

### Key Principles:

- The right cache layer depends on your deployment topology, not just your data
- In-process cache is invisible to other instances — fine for single-server, broken for load-balanced
- Distributed cache adds a network hop but makes the cache a shared resource

### Examples:

- Single server with an in-process dict cache: works perfectly, zero overhead
- Load balancer with two instances, each with their own in-process dict: user A gets instance 1 (fresh data), user B gets instance 2 (stale data after an update) — inconsistent

## 02.1 Cache Placement

### Learning Objectives:

- Choose between in-memory, Redis, and CDN cache based on deployment needs
- Understand what is gained and lost at each layer

### Content Outline:

- In-memory (app): Python dict, `functools.lru_cache` — zero latency, no network, lost on restart, not shared
- Distributed (Redis): shared across all instances, survives restarts, adds a small network hop
- CDN/HTTP cache: `Cache-Control` headers, handled by infrastructure, for public responses only

**Transitions:** Now that we know where to put the cache, the next lesson tackles what happens when the data it holds becomes outdated.

### Key Principles:

- Use in-memory for single-instance development or non-critical caching
- Use Redis in production with multiple instances
- Use CDN only for public, non-authenticated responses

### Examples:

- `functools.lru_cache` on a pure function: zero setup, immediate benefit, disappears when process restarts
- Redis `SET user:42 <data> EX 300`: shared across all API instances, survives a pod restart

- **Anti-pattern — In-Memory Cache Behind a Load Balancer:** Two instances of your API each maintain their own dict cache. You update a record, invalidate the cache on instance 1 — but instance 2 still serves stale data. The fix: use Redis so all instances share the same cache state.
- 

## 02.2 Cache Invalidation

### Learning Objectives:

- Explain why cache invalidation is considered one of the hardest problems in computer science
- Implement TTL-based and event-based invalidation and state when to use each

### Content Outline:

- The core problem: the cache is a copy — when the original changes, the copy is wrong
- TTL (Time-To-Live): the cache entry expires after N seconds; simple, predictable, occasionally stale
- Event-based invalidation: when the underlying data changes, explicitly delete or update the cache key; always accurate, more complex
- The tradeoff: TTL is simpler but can serve stale data; event-based is precise but requires coordination

**Transitions:** Now that we have a mental model for where cache lives and how to keep it fresh, Section 03 covers the four established patterns for reading and writing to the cache.

### Key Principles:

- Every cached value has an expiry — explicit (TTL) or implicit (event)
- TTL is appropriate when slight staleness is acceptable
- Event-based is appropriate when stale data has real consequences

### Examples:

- TTL: `cache[key] = (value, time.time() + 60)` — expires in 60 seconds regardless of what happens to the data
  - Event-based: when a product's price is updated via `PATCH /products/42`, the handler explicitly deletes `cache["product:42"]`
  - **Anti-pattern — No Invalidation:** Setting a cache entry with no TTL and no deletion logic. The data served by the cache drifts further from reality every day until the server restarts. The fix: every cache entry must have either a TTL or an explicit invalidation trigger.
- 

## 03.0 Caching Strategies

### Learning Objectives:

- Understand why named caching strategies exist — different read/write patterns need different approaches
- Distinguish the four strategies by who populates the cache and when writes reach the database

### Content Outline:

- The question strategies answer: given a read or write request, what happens to the cache and the DB, and in what order?
- Cache-Aside: the application manages the cache manually on reads
- Write-Through: every write goes to cache and DB simultaneously
- Write-Behind: write to cache first, sync to DB asynchronously
- TTL-Based: use expiration as the sole invalidation mechanism for any of the above

**Transitions:** 03.1 covers each strategy in detail — mechanics, when to use it, and what can go wrong. 03.2 then implements Cache-Aside in Python.

### Key Principles:

- There is no universally best strategy — the right choice depends on read/write ratio and consistency requirements
- Cache-Aside is the most common starting point for read-heavy APIs
- Write-Behind should never be used when data loss is unacceptable

### Examples:

- A product catalog (read-heavy, rarely updated) → Cache-Aside is natural
- A financial ledger (every write must be durable) → Write-Behind is dangerous here
- A public leaderboard (acceptable to be 30 seconds stale) → TTL-Based is simple and sufficient

## 03.1 The Four Strategies

### Learning Objectives:

- Describe the mechanics of Cache-Aside, Write-Through, Write-Behind, and TTL-Based caching
- Select the appropriate strategy given a scenario's consistency and performance requirements

### Content Outline:

- **Cache-Aside (Lazy Loading):**
  - Read: check cache → miss → load from DB → store in cache → return
  - Write: write to DB only; cache is invalidated or left to expire
  - Best for: read-heavy workloads, data that isn't always needed
- **Write-Through:**
  - Every write goes to cache AND DB in the same operation
  - Cache is always consistent with the DB
  - Best for: data that is read immediately after being written
  - Cost: every write touches both stores — slightly slower writes
- **Write-Behind (Write-Back):**
  - Write to cache first; a background process syncs to DB asynchronously
  - Very fast writes; risk: if the process crashes before syncing, data is lost
  - Best for: high-throughput write scenarios where eventual consistency is acceptable
- **TTL-Based:**
  - No explicit invalidation; cache entries simply expire after N seconds
  - Can be combined with any of the above

- Best for: data with a known acceptable staleness window

**Transitions:** Now that all four strategies are understood, 03.2 implements Cache-Aside — the most common pattern — in Python.

### Key Principles:

- Cache-Aside keeps the app in control — it decides when to populate and when to invalidate
- Write-Through keeps cache and DB always in sync at the cost of write speed
- Write-Behind maximizes write speed but introduces data loss risk
- TTL-Based is the simplest invalidation mechanism and pairs well with any strategy

### Examples:

- **Anti-pattern — Write-Behind for Financial Data:** A payment service writes to cache first and syncs to DB asynchronously. The server crashes before the sync. The payment is lost with no record. The fix: for data where loss is unacceptable, use Write-Through or skip caching writes entirely.
  - Cache-Aside illustrated: `get_user(42)` → check `cache["user:42"]` → not found → query DB → `cache["user:42"] = (result, now + 300)` → return result
- 

## 03.2 Cache-Aside in Practice

**Challenge Prompt:** Implement a `get_user` function that uses Cache-Aside with a 5-second TTL: on a cache hit print `CACHE HIT` and return the cached value; on a cache miss print `CACHE MISS`, simulate a DB call, store the result, and return it.

### Solution Code:

```
import time

cache = {}

def get_user(user_id: int) -> dict:
    now = time.time()
    if user_id in cache:
        value, expires_at = cache[user_id]
        if now < expires_at:
            print("CACHE HIT")
            return value
    print("CACHE MISS")
    result = {"id": user_id, "name": f"User {user_id}"}
    cache[user_id] = (result, now + 5)
    return result

print(get_user(1))
print(get_user(1))
print(get_user(2))
```

## Challenge Code:

```
import time

cache = {}

def get_user(user_id: int) -> dict:
    now = time.time()
    # If the key exists in cache and the TTL hasn't expired, return it
    # your code here

    # Cache miss: simulate a DB call
    print("CACHE MISS")
    result = {"id": user_id, "name": f"User {user_id}"}
    # Store result in cache with a 5-second TTL
    # your code here
    return result

print(get_user(1))
print(get_user(1))
print(get_user(2))
```

---

## 04.0 Caching Knowledge Check 🧠

### Learning Objectives:

- Demonstrate understanding of caching fundamentals, architecture, invalidation, and strategy selection

**Question Format:** Multiple choice

### Questions:

1. What is a cache hit?
  - a) The cache key was not found and the DB was queried
  - b) The requested value was found in the cache and returned without touching the DB ✓
  - c) The cache was cleared due to a TTL expiry
  - d) A write operation updated both the cache and the DB
2. Your API serves a product catalog that is updated once a day but queried 10,000 times per hour. Should you cache it?
  - a) No — the data changes, so caching will always serve stale results
  - b) No — caching is only for static data
  - c) Yes — high read frequency with low write frequency is the ideal caching candidate ✓
  - d) Yes — but only if the response is under 1KB



3. You deploy your FastAPI app behind a load balancer with three instances. Which cache type will cause inconsistency between instances?
- a) Redis
  - b) A CDN
  - c) An in-process Python dict ✓
  - d) A database query cache
4. What is the main risk of event-based cache invalidation compared to TTL?
- a) It always serves stale data
  - b) It requires more implementation effort and coordination ✓
  - c) It uses more memory than TTL
  - d) It cannot be used with Redis
5. Which caching strategy writes to both the cache and the database in the same operation?
- a) Cache-Aside
  - b) Write-Behind
  - c) TTL-Based
  - d) Write-Through ✓
6. A developer stores a user's authentication token in a shared CDN cache keyed only by the endpoint path. What is the risk?
- a) The token will expire too quickly
  - b) Other users requesting the same endpoint may receive another user's token ✓
  - c) The cache will run out of memory
  - d) The CDN will reject the token format
7. You are building a high-throughput event logging service. Writes happen thousands of times per second, and losing a small number of events is acceptable. Which strategy fits best?
- a) Write-Through — always consistent
  - b) Cache-Aside — reads only, writes go straight to DB
  - c) Write-Behind — fast writes with async DB sync ✓
  - d) TTL-Based — expiry handles everything
8. In Cache-Aside, who is responsible for populating the cache?
- a) The database driver
  - b) A background sync process
  - c) The application code, on cache miss ✓
  - d) The CDN on every request

#### Evaluation Criteria:

- 7–8 correct: solid understanding of caching concepts and strategy selection
- 5–6 correct: good foundation; review cache architecture and strategy tradeoffs
- Below 5: revisit sections 02 and 03 before applying caching in production code

**Score Interpretation:** Each question maps to a core concept: Q1 (fundamentals), Q2 (when to cache), Q3 (placement), Q4 (invalidation), Q5–Q7 (strategies), Q8 (Cache-Aside mechanics).

---

## 05.0 Outro

### Content Outline:

- Course recap: what students accomplished
  - Understood what caching is and how the hit/miss cycle works
  - Learned to evaluate whether caching is worth it for a given data access pattern
  - Mapped the three cache placement options (in-memory, Redis, CDN) to real deployment scenarios
  - Understood TTL and event-based invalidation and their tradeoffs
  - Learned the four caching strategies and when to apply each
  - Implemented Cache-Aside with TTL in Python
- Connection to bigger picture: caching is one layer in the performance stack — working alongside serializers (what you send) and payload shaping (how much you send) to make APIs fast under load
- Next steps: explore Redis in depth, learn about cache stampede and how to prevent it, look into HTTP caching headers ([Cache-Control](#), [ETag](#)), and distributed cache patterns for microservices
- Resources: Redis documentation, Martin Fowler's caching patterns, Python [functools.lru\\_cache](#) docs
- Encouragement: caching feels complex at first because of invalidation — but Cache-Aside with a reasonable TTL solves 80% of real-world cases. Start simple, measure, then add complexity only when needed.

**Note:** This is conclusion only — no theory, exercises, or assessment.

---